

Arquitetura do Projeto

LocadoraVeiculosApi

Documento técnico com foco na organização arquitetural, responsabilidades das camadas e estrutura MVC limpa aplicada ao projeto de locadora de veículos.

ASP.NET Core • Entity Framework Core • SQL Server Express

Objetivo deste documento

- explicar a arquitetura adotada no projeto;
- descrever a organização por camadas e responsabilidades;
- apresentar a estrutura de pastas recomendada para um MVC limpo;
- registrar o fluxo principal da aplicação e os próximos passos de evolução.

1. Visão geral da arquitetura

O projeto foi organizado como uma API REST em ASP.NET Core, com persistência em SQL Server Express e mapeamento relacional via Entity Framework Core. A proposta arquitetural não busca complexidade excessiva; o objetivo é manter uma base acadêmica simples, mas bem separada em responsabilidades.

No contexto deste sistema, o padrão MVC é aplicado da seguinte forma: os Controllers recebem as requisições HTTP, as entidades do domínio representam o núcleo do Model, e as respostas JSON, o Swagger e a página mínima de testes funcionam como interface de apresentação da aplicação.

Decisão arquitetural: manter um MVC limpo e pragmático, com separação entre Controllers, Application, Domain, Infrastructure e Shared, sem introduzir complexidade desnecessária para o escopo do trabalho.

2. Organização por camadas

A estrutura proposta para o projeto foi definida para deixar a solução mais legível, testável e fácil de manter. Cada camada possui uma responsabilidade específica:

Camada	Responsabilidade
Controllers	Receber requisições HTTP, expor endpoints da API e devolver respostas adequadas.
Application	Concentrar DTOs e espaço para Services e Interfaces, responsável pela orquestração da aplicação.
Domain	Manter as entidades centrais do sistema e o modelo conceitual do domínio.
Infrastructure	Conter DbContext, migrations e integração com o banco de dados.
Shared	Reunir recursos transversais, como middleware e helpers reutilizáveis.
Views	Disponibilizar um frontend mínimo para testes rápidos da aplicação.

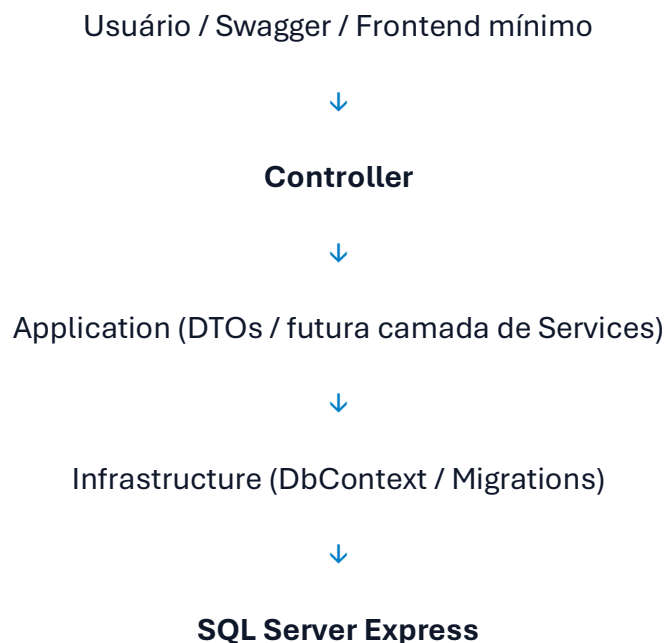
3. Estrutura de pastas adotada

Abaixo está a estrutura principal do projeto após a reorganização arquitetural:

```
LocadoraVeiculosApi/
├── Controllers/
│   ├── HomeController.cs
│   ├── AlugueisController.cs
│   ├── CategoriasVeiculoController.cs
│   ├── ClientesController.cs
│   ├── FabricantesController.cs
│   ├── FiltrosController.cs
│   └── VeiculosController.cs
├── Application/
│   ├── DTOs/
│   ├── Interfaces/
│   └── Services/
├── Domain/
│   └── Entities/
└── Infrastructure/
```

```
| | Data/
| | Migrations/
| Shared/
| | Helpers/
| | Middleware/
| Views/
| | Home/
| Program.cs
| appsettings.json
| appsettings.Development.json
| LocadoraVeiculosApi.csproj
| CRUD.sln
```

4. Fluxo principal da aplicação



Esse fluxo resume a ideia central do projeto: a aplicação recebe requisições, trata os dados nos controllers, trabalha com entidades e DTOs bem organizados, persiste no banco por meio do Entity Framework Core e devolve respostas HTTP padronizadas.

5. Mapeamento dos componentes atuais

Componente	Papel arquitetural
Controllers/	Camada de entrada HTTP da aplicação
Application/DTOs/	Contratos de entrada e saída usados pelos endpoints
Domain/Entities/	Entidades do sistema: Cliente, Fabricante, CategoriaVeiculo, Veiculo e Aluguel
Infrastructure/Data/	ApplicationContext e ApplicationContextFactory
Infrastructure/Migrations/	Histórico e snapshot do banco
Shared/Helpers/	Tratamento auxiliar de exceções do banco
Shared/Middleware/	Middleware global para tratamento de erros
Views/Home/	Interface mínima para testes rápidos da aplicação

6. Benefícios da estrutura aplicada

- melhor separação de responsabilidades entre entrada HTTP, domínio e persistência;
- maior clareza para manutenção e evolução do projeto;
- facilidade para migrar parte da lógica dos controllers para Services futuramente;
- estrutura mais adequada para apresentação acadêmica e leitura do código;
- base pronta para crescer sem perder organização.

7. Próximo passo recomendado

A reorganização atual melhora a estrutura física do projeto e prepara o terreno para uma evolução importante: mover a lógica de negócio hoje presente nos controllers para uma camada de Services

em Application/Services. Isso deixará os controllers mais enxutos e reforçará ainda mais a proposta de um MVC limpo.